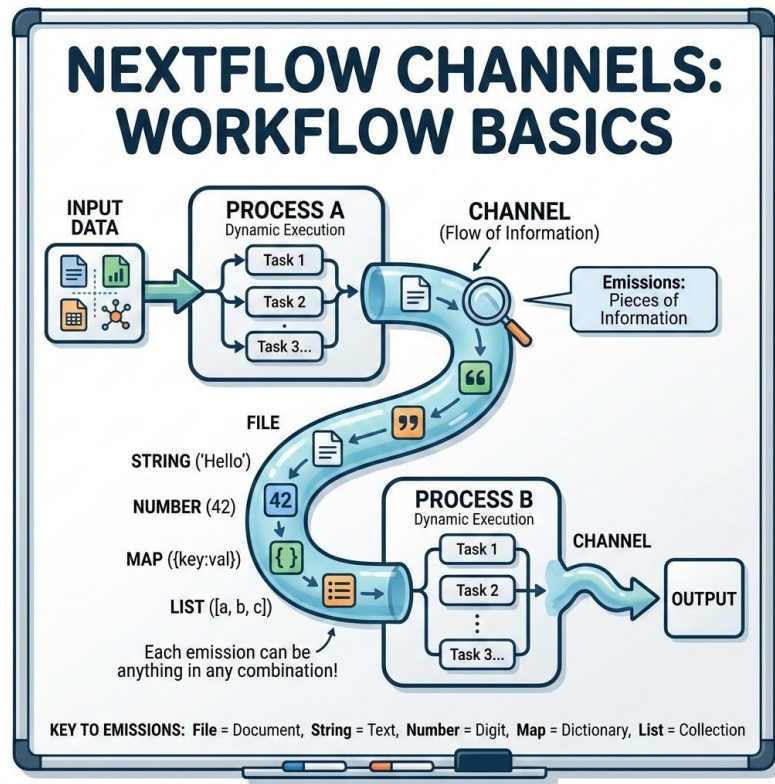


Hello Nextflow

Code introduction: channels and operators

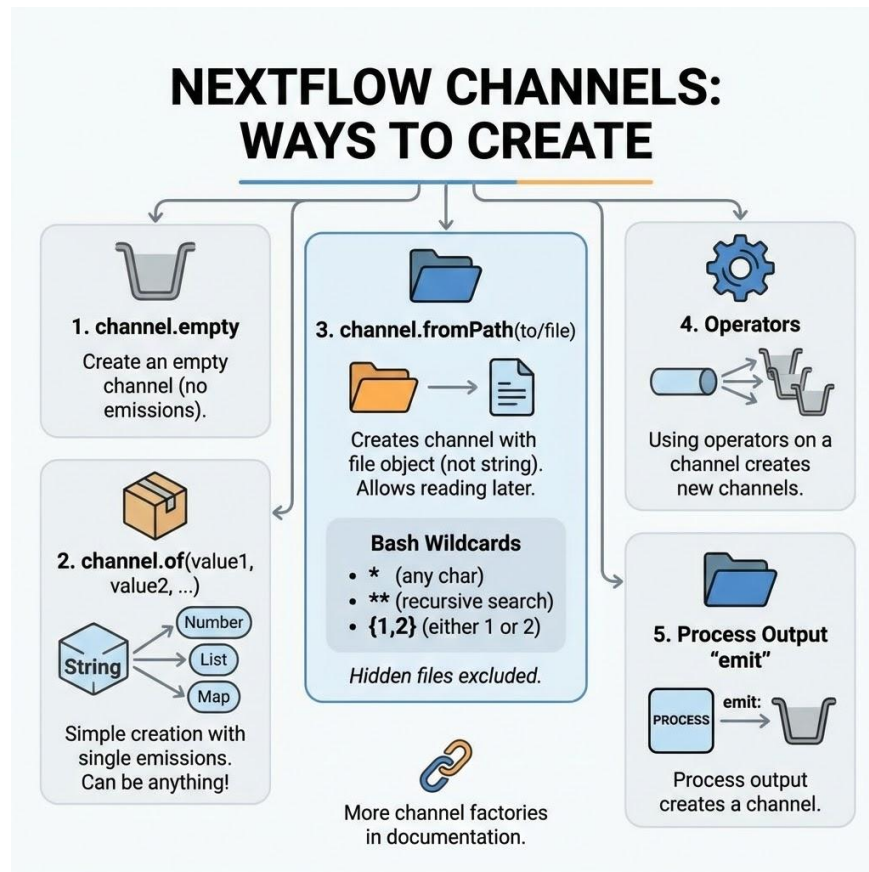
Connect your processes in a workflow: channels

- A pipeline, is a series of steps represented by processes where each task is executed a dynamic number of times based on the input data
- To connect the different processes together in an actual workflow, Nextflow uses **channels** → [Workflows — Nextflow documentation](#)
- Channels are “flows” of information; each “piece of information” that pass through a channel is called “**emission**”
 - As a minimal example, if you have to pass a file from a process “A” to a process “B”, the emission will consist of the file itself
 - Each emission can be anything: a string, a number, a file, a map (dictionary), a list of the previous in any combination you like!



Creating a new channel: channel factories

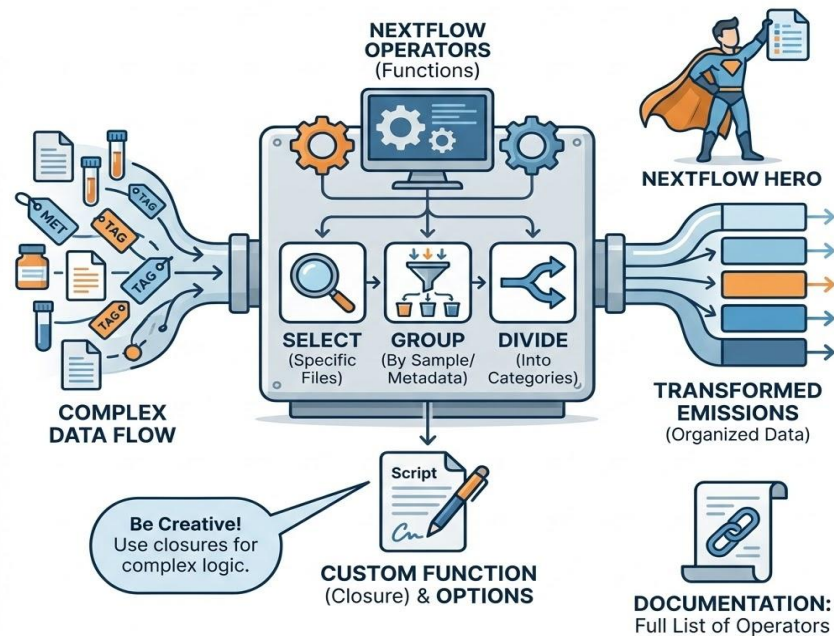
- There are multiple ways to create a `channel`:
 - `channel.empty`: create an empty channel (e.g. with no emissions)
 - `channel.of(string, value, ..., list_of_things)` → this is the most simple way to create a channel with emissions; here each value will be a single emission
 - You can be creative: elements can be anything (strings, numbers, maps, lists...)
 - `channel.fromPath(/path/to/file/to/read.txt)`: this will create a channel with a given file as emission → the file will not be treated as a string but as a file object which allows you to read it later using operators
 - You can be creative also here, creating emission with list of files
 - You can use Bash wildcards to match different files:
 - `*` will match any character
 - `**` will look recursively in the specified dir → be careful with large directories
 - `{1,2}` will look for either "1" or "2" in the given expression
 - Hidden files are excluded from the search
 - Operators: using operators on a channel will create one or more channels
 - Process output `emit`: using this option on a given defined process output will create a channel
- There are other channel factories, have a look here → [Channel factories — Nextflow documentation](#)



Working with channels: operators

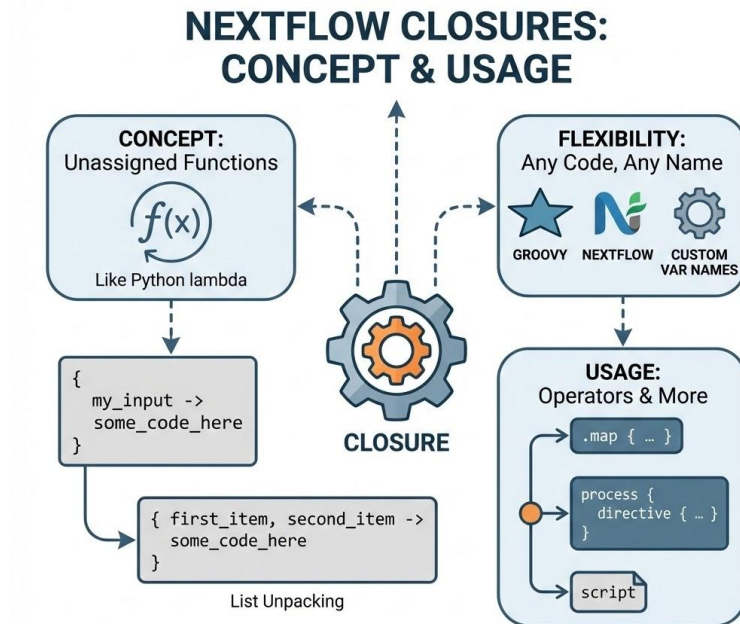
- In more complex (and realistic) scenarios, you don't simply pass a file from a process to the next one
 - You may need to select specific files, group files by sample/condition/metadata, divide them into categories etc.
- In Nextflow, you can apply functions to channels to **change the content of each emission**: these functions are called **operators**
- There are many different **operators** and learning how to play and be creative with them is what will make you one day a Nextflow hero
- Since operators are functions, they have options or you can pass custom function (**closures**) to them
- You can find here the list of all the possible operators → [Operators — Nextflow documentation](#)

NEXTFLOW CHANNEL OPERATORS: TRANSFORMING YOUR WORKFLOW



Closures: custom unnamed functions

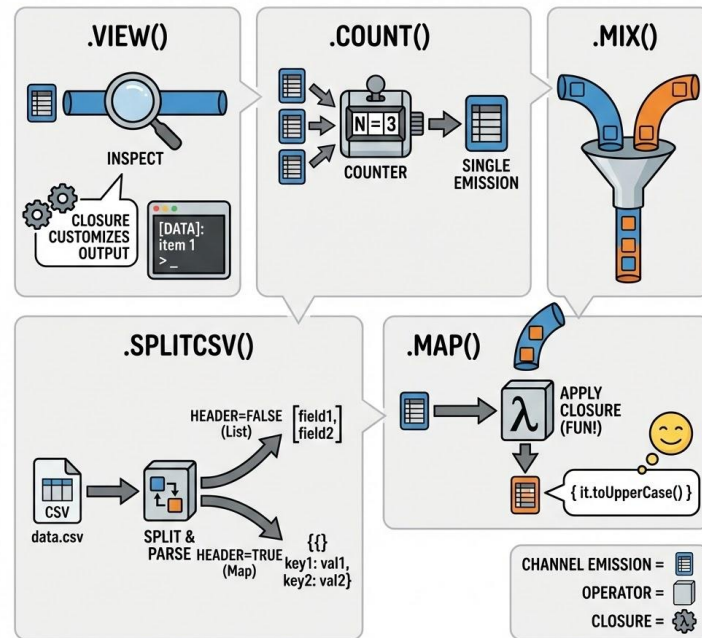
- Closure are functions that you don't assign to a variable
 - For those familiar with python, closure look like the python `lambda` functions
- A closure is defined using the `{my_input -> some_code_here}` notation
 - If your input is a list, you can unpack it: `{firsrt_item, second_item -> some_code_here}`
 - You can name your input variables as you like and you can use any piece of groovy and Nextflow code inside them
- Closures can be used in many places into Nextflow, here we will look at them specifically in operators but they can be used, for example, to define dynamical directives for the processes
- Full closure reference → [Scripts — Nextflow documentation](#)



Basic operators: `.view()`, `.count()`, `.mix()`, `.splitCsv()`, `.map()`

- `.view()`
 - used to inspect each emission of a channel
 - a closure can be used to customize the visualization output
- `.count()`: count the number of the emission → from n emission to a single one
- `.mix()`: mix together the emissions coming from two channels → from 2 channels to 1
- `.splitCsv()`
 - The operator takes a single emission (a csv file) and:
 - create a number of emissions equal to the number of rows in the file
 - split the row string into elements using the “,”
 - The resulting emission is a list of elements → each element is a field of the csv
 - The `.splitCsv()` operator have the `header` option (boolean)
 - If true, instead of a list of fields it is returned a map using the header for defining the keys
- `.map()`: apply a custom function (have fun with closures!) to each emission

NEXTFLOW CHANNEL OPERATORS: SUMMARY



Create your initial channel from a csv/tsv file

- A very typical scenario in bioinformatics is to create a channel starting from an input file, generally in csv or tsv format
 - By using a the `channel.fromPath` channel factory and few operators you can have your input set up and ready to be used in your processes
- You will need 2 operators to achieve that result:
 - `.splitCsv()` → used to read the csv file
 - `.map()` → used to rearrange the file content to have the emission customized to your needs
- Tip: for complex workflows, be consistent on your channel/emission structure!
 - Usually, channel emissions will be tuples of elements where the first one is collecting your metadata and all the others will be files linked to that metadata
 - `tuple(meta, file_1, file_2, ... file_n)`
 - The element to collect the metadata can be a list or, better, a map (dictionary)

Let's get our hands dirty!

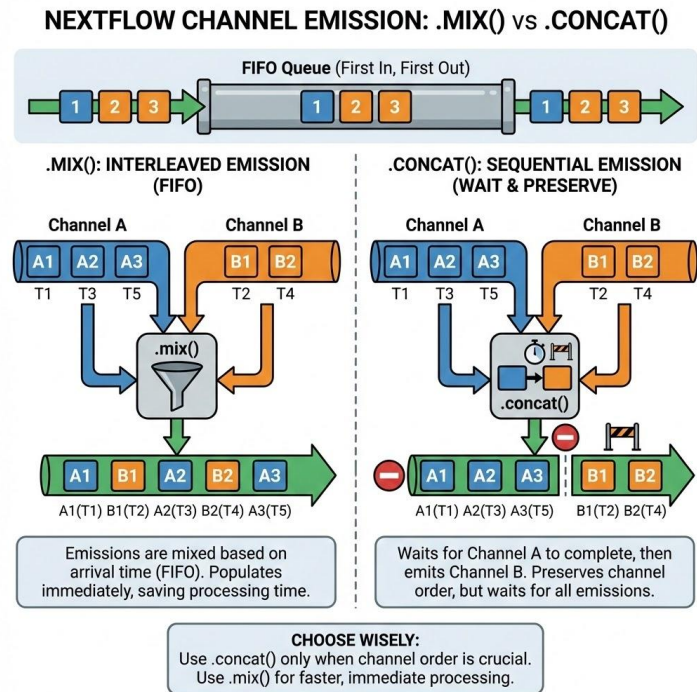
- You will find this first batch of exercises in the repository at the following path:

`practicals/day1/2_basic_workflow/`

- Here you will find a README with your exercises
 - For each exercise, you will find a “initialised” version of the task if you need some hint and the solution
- The input files needed are located in that directory
 - metadata.csv
 - metadata.tsv
 - nextflow.config → modify this file with the correct path to the csv and tsv files
 - In alternative you can use the params.yaml to define your parameters

Emission order in channels: `.mix()` and `.concat()` difference

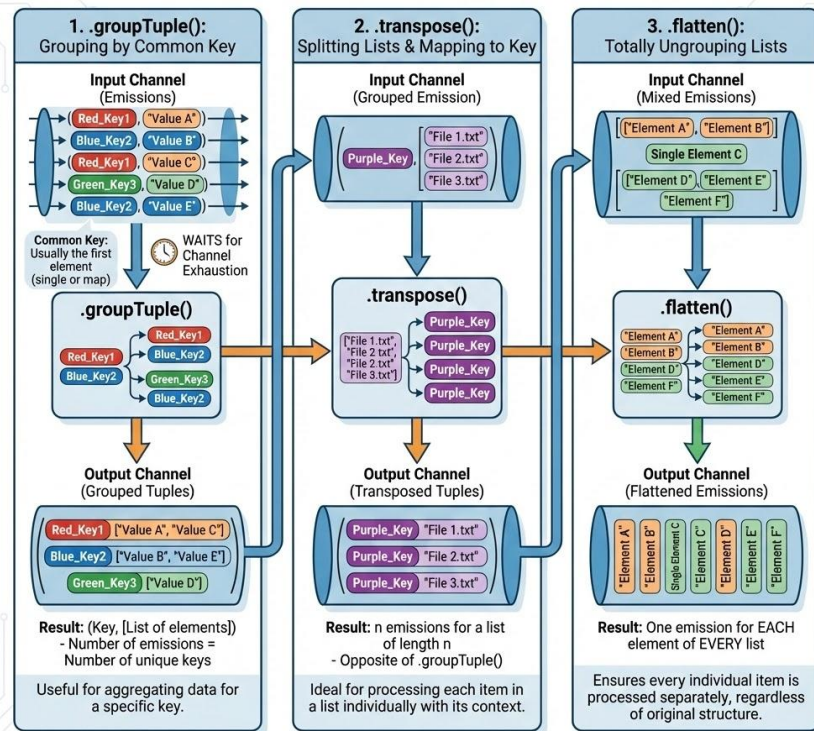
- Channels are “FIFO queues” → first in, first out
 - The order of the emissions is determined by the order of the events in the workflow: if a sample will finish a process sooner than another, it will be emitted first
- You should account this concept since there are some operators that “wait” until each emission has passed through a channel before being executed
- An easy example are the `.mix()` and `.concat()` operators
 - Both operators create a single channel taking the emissions of two different channels
 - The `.mix()` operator will result in a channel where the emissions of both channels are actually mixed together following the FIFO concept
 - The `.concat()` operator will wait for both channels to be exhausted (e.g. for them to have all the emission done) and then will concat the two channels, preserving the order of the first and the second channels
 - Given their different behaviour, use `.concat()` only when you actually need to preserve the channel order, otherwise the resulting channel will be created only when all values are emitted → using `.mix()` allow you to populate the resulting channel as soon as an element is emitted, thus saving processing time



Group and ungroup: `.groupTuple()`, `.transpose()` and `.flatten()`

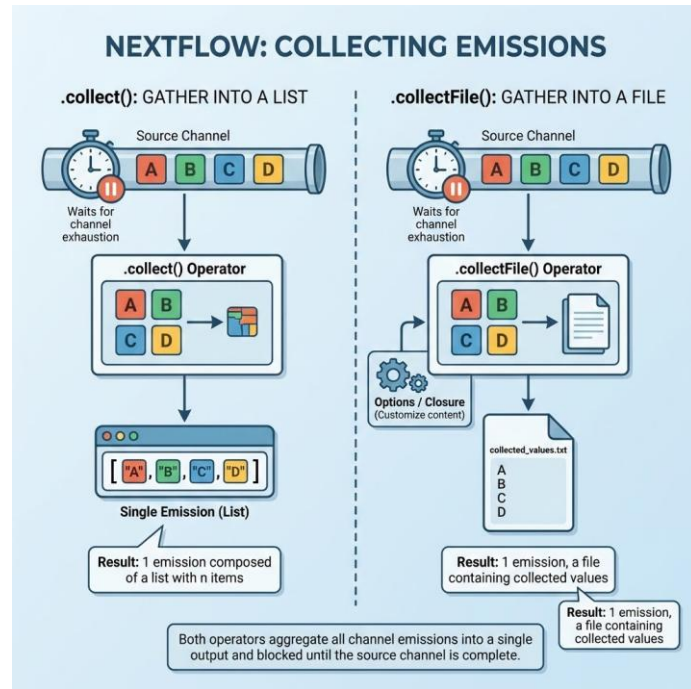
- To group emissions based on a common key, you can use the `.groupTuple()` operator
 - The common key is usually the first element of a tuple emission and it can be a single metadata or a map of multiple metadata together
 - The number of emissions will be equal to the number of unique keys
 - The resulting emission will be a tuple where you have your common key and a list of grouped elements → the length of this list will be equal to the number of elements that share that key
 - As for the `.concat()` operator, also this operator waits until the source channel is exhausted
- To split a list of files and map it to a common key, you can use `.transpose()`
 - In this case the result it will be the opposite: from a single emission where you have a tuple with a first element and then as second element a list of length n, you will obtain n emissions with just the first tuple item and one element of the list
- To totally ungroup each possible list you have in your emissions, you can use `.flatten()` which will return one emission for each element of each list

NEXTFLOW GROUPING & UNGROUPING OPERATORS: `.groupTuple()`, `.transpose()`, `.flatten()`



Collecting emissions in a single one: `.collect()` and `.collectFile()`

- If you need to collect all the emissions of a channel into a single emission, you can use the `.collect()` operator
 - The resulting emission will be always 1 composed by a list with n items where n is the number of emission of the source channel
- The `.collectFile()` operator does the same thing as `.collect()` but instead of emitting a list, will emit a file where the content of the file will be the collected values
 - You can customize how to write the content of the file using options and/or a closure
- Both operators will wait for the source channel to be exhausted



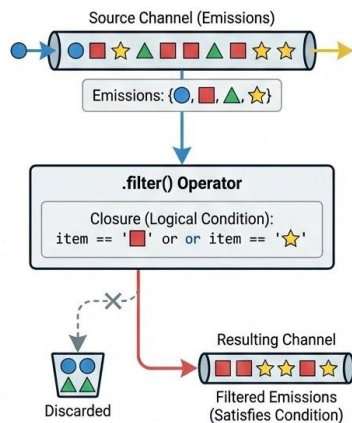
Subset using logical conditions: `.filter()` and `.branch()`

- When you need to filter the emissions of a channel, you can use the `.filter()` operator
 - You will need a closure to define the logical conditions for filtering
 - The resulting channel will contain only the emissions that satisfy the filter condition
- If you need to instead divide your emission in many different channels, then `.branch()` is what you are looking for
 - As for `.filter()` you need a closure to define a logical condition, here you will need a closure too but you can set up any named conditions
 - The incoming emissions will be divided into the different categories you set up in the code
 - In this way, starting from one channel you end up with multiple channels
 - Some of these may be potentially empty

NEXTFLOW FILTER OPERATORS:

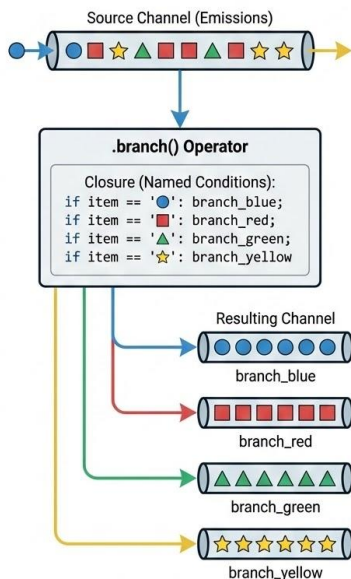
Subset with Logical Conditions

`.filter()` Operator: Single Channel Filtering



Closure defines condition.
Channel contains ONLY satisfying emissions.

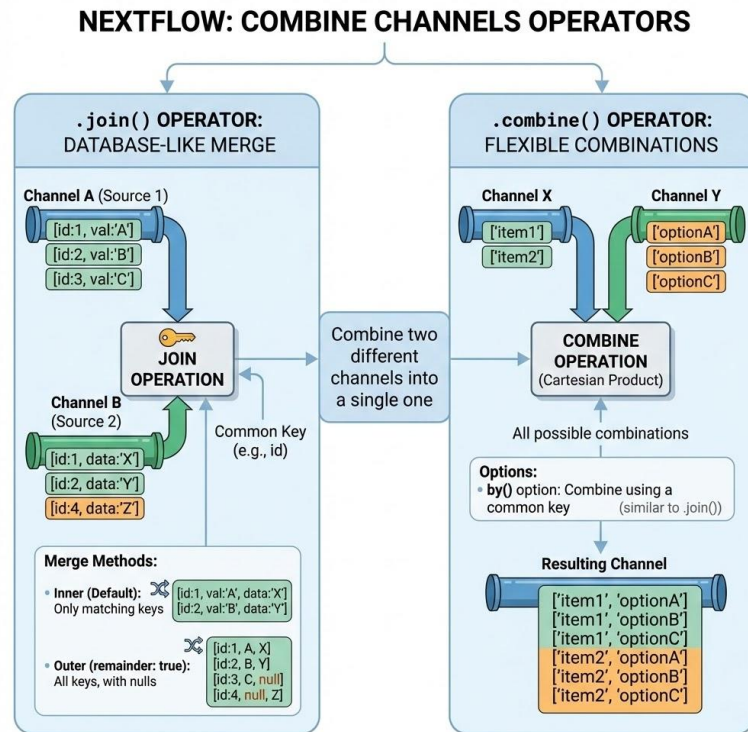
`.branch()` Operator: Multi-Channel Division



Closure with multiple named conditions.
One channel divided into multiple resulting channels.
Channels can be potentially empty.

Combine channels: `.join()` and `.merge()`

- When you want to merge the information of two different channels into a single one, you will use either `.join()` or `.combine()`
- You can have different ways of merging channels
 - With `.join()`, you can look at this operation as you would do for merging two different databases
 - You need a common key on which to perform the merge
 - There are different methods for merging: inner (default) or outer (`remainder` option)
 - With `.combine()` you are more flexible and you can have all possible combinations of events (cartesian product)
 - You can filter the resulting channel using the `by()` option to combine by using a common key

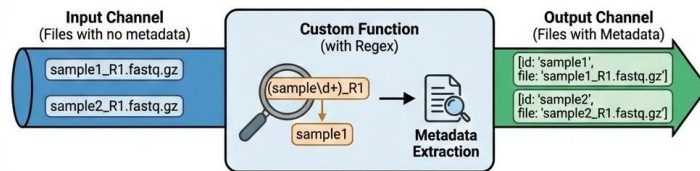


Advanced .map() and .multiMap()

- For most of the custom things, the `.map()` operator is the one that will solve your issues since it allow to apply any function to your channel
 - For using the previously described operators, you may need to rearrange/reorganize/add new metadata in your emissions
 - Among the many different thing you could do in map, a useful one is to use regular expressions on files to fetch patterns in file names and extract metadata from these
 - For example, if you have a file with no attached metadata but you know that the name of the file is the sample it represent, then you can “extract” the sample name from the file name
- If you need to apply different functions to the same input channel, then you can use `.multiMap()` instead of using `.map()` multiple times
 - This is useful if you have one source channel that you need to rearrange in different ways for different purposes
 - From a single channel you will end up with many different named channels but the number of emissions of each of these will be the same as the input channel

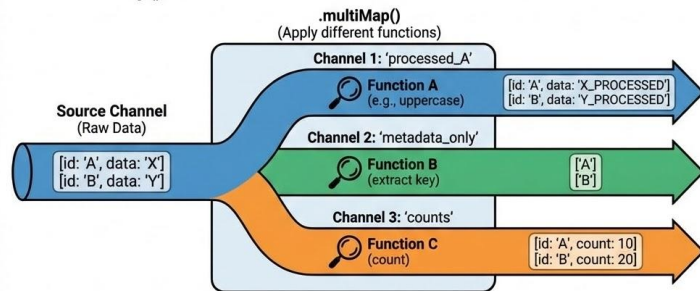
NEXTFLOW ADVANCED OPERATORS: .map() & .multiMap()

.map() OPERATOR: CUSTOM TRANSFORMATIONS & METADATA EXTRACTION



Apply any function to rearrange, reorganize, and add metadata.
Use regex to extract information from filenames.

.multiMap() OPERATOR: SPLIT & TRANSFORM ONE SOURCE TO MANY

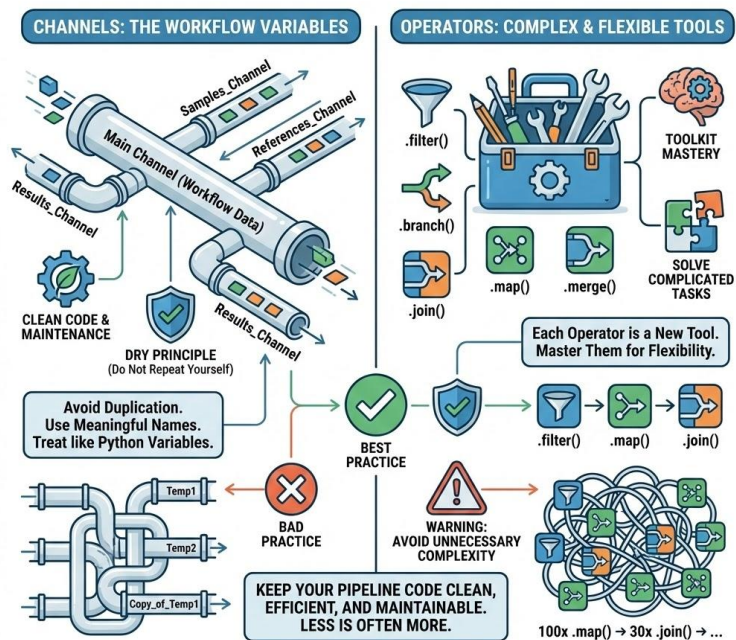


Apply different functions to the same input.
Results in multiple named channels with the same number of emissions.

Key concepts and summary

- Channel represents the main variables of your workflows
- Using channels properly, avoiding duplication of information and assigning meaningful names/content to them will keep your pipeline code clean and easy to maintain
 - You should work with channels as any other variable in your python scripts: try to follow the “do not repeat yourself” (DRY) principle
- Operators are the most complex/flexible piece of code you will find in Nextflow
 - Every operator that you will know and master will give you an additional tool to solve complicated tasks that you will find while developing your pipelines
 - As for the channels, follow the DRY principle and avoid using too many unnecessary operators to keep your code clean
 - `100 .map()` operations on a channel followed by `30 .join()` maybe are not that necessary

NEXTFLOW CHANNELS & OPERATORS: KEY TAKEAWAYS



Let's play with operators!

- You will find this second batch of exercises in the repository at the following path: `practicals/day1/4_operators_playground/`
 - Here you will find a README with your exercises
 - For each exercise, you will find a “initialised” version of the task if you need some hint and the solution
 - The input files needed are located in that directory
 - `metadata.csv`
 - `nextflow.config` → modify this file with the correct path to the csv and tsv files
 - In alternative you can use the `params.yaml` to define your parameters
 - You will also find a file named `hello-nextflow.nf` where all the exercise solutions are written in a unique workflow → do not cheat, look at it only after the exercise section ;)